

MANTIS presentation — read-aloud script

Deck: MANTIS_Presentation_2026-09-24.pptx · 15 slides · about 8 minutes at a relaxed pace (shortened version).

Read each block as written; it is in spoken language. The same text is in the speaker notes of each slide. If you are short on time, skip slides 7 and 12 and go straight to the demo.

Slide 1 — MANTIS

About 30 seconds

Thanks for having us. This is MANTIS: a testbed where you attack a bank of AI agents on purpose, and know exactly what happened. In the next few minutes: what it is, how it's built, the attacks and results, how a new person gets started, and a short live demo. Everything runs on a realistic banking system with synthetic data.

Slide 2 — What MANTIS is

About 30 seconds

Banks are splitting work across specialist AI agents, so every hand-off between agents is a new place for an attack or a failure. MANTIS tests that on purpose. One: the system under test is a real banking multi-agent system, and we never modify it. Two: an experiment is just a YAML file. Three: every run leaves evidence and an automatic verdict. To be clear, it's a testbed. It measures what an attack did. It is not a detector or a defense product.

Slide 3 — Architecture

About 30 seconds

The architecture is simple. Everything an agent does passes one of five checkpoints: request in, agent start, model call, tool action, result out. They sit on a hook bus, and any plugin can watch or change what passes. Attacks inject a fault, failures simulate ordinary faults, defenses deny or redact, and observability records everything, always last. At each checkpoint a plugin can continue, mutate, deny, error, skip or delay. The key point: no banking code changes to run an experiment.

Slide 4 — What has been built

About 30 seconds

Here is what's been built from the start. The foundation was nine work packages: a frozen baseline of the original system with 49 regression tests, a modular runtime, YAML configuration, the hook bus, an observability pipeline, the first attack and fault plugins, scoring and benchmarks, a command-line workflow, and tests, docs and an open-source release. Since then: more exporters, three new banking workloads, a second institution, six defenses, a browser UI, and an extended scenario library. Seven hundred seventy offline tests back it.

Slide 5 — The system under test

About 30 seconds

This is what we test against. A root agent routes each request to a front, mid or back office. Front office is customer-facing: transaction monitoring, chatbot, disputes. Mid office is operations planning, representative assist and loan pre-approval. Back office is end-of-day reconciliation and SAR escalation. That's 34 agents and 27 tools on a real banking service, so when a transfer runs, real state changes. Data is synthetic.

Slide 6 — An experiment is a YAML file

About 15 seconds

A complete experiment: the scenario, the attack plugin, where it attaches, and what a correct run looks like. This one hijacks a routing decision, so a suspicious transaction is quietly sent to the ordinary customer-service path. Four commands drive everything: validate, run, evaluate, and campaign to sweep a folder.

Slide 7 — What every run produces

About 30 seconds

Every run leaves a folder of evidence: a manifest that fingerprints the config and the model, a full trace, hook coverage, and a seven-dimension scorecard. The idea to take away: fired is not the same as succeeded. Here an attack rewrote a small transfer into five thousand dollars. It fired. The bank's own validation rejected it, so no money moved. We report those as two separate answers.

Slide 8 — The attack and fault catalog

About 30 seconds

Five families. Prompt injection adds an instruction to an agent's model request. Message spoofing forges a message from a trusted colleague agent. Route confusion hijacks a hand-off. Tool mutation rewrites a real action's arguments just before it runs. And reliability faults simulate delays, timeouts and corrupted results. We kept faults separate from attacks on purpose. The extended library adds 26 variants across all three domains.

Slide 9 — Results: the original five families

About 30 seconds

Each family was run five times against the live system. Prompt injection and message spoofing reached the model every time, and it resisted. The transfer mutation fired every time and the bank's own validation stopped it. Two results stand out: route confusion diverted a suspicious transaction away from compliance review every time, and the ledger

attack posted to the wrong batch unnoticed. Faults were handled safely and logged as anomalies. All on gpt-oss-20b (open-weight), served through UF Navigator.

Slide 10 — Six defenses on the same framework

About 30 seconds

Defenses use the same mechanism as attacks, so you can compare them equally. We built six, covering all five categories of security mechanism in the plan. The rule is to re-check backend truth rather than trust the model. An amount limit, a routing guard that can redirect back to the compliant path, a batch integrity guard, response redaction, a rate limit, and action isolation. All six were verified live. This covers the gaps we measured; it is not a comprehensive defense suite.

Slide 11 — The extended scenario library

About 60 seconds

To widen coverage we built a library on the same architecture: 19 measured baselines and 26 attack variants, 123 scored live trials, each starting from a fresh database and recording what changed. Four things stood out. Where an attack lands matters: the same payload changed the outcome in 1 of 5 trials at the monitor, but bypassed manual review in 5 of 5 at the decision agent. Silence can be the outcome: the SAR agent filed nothing 5 of 5 times. Data-level effects are invisible to tool-use scoring: in 20 of 20 trials the database changed while it flagged 0, which is why we record the database. And faults propagate through workflows. Read these as what happened in this sample, five trials per configuration, one model.

Slide 12 — Getting started

About 30 seconds

Getting started is five steps. Clone and pip install. Add a model key to the environment file, or skip it: mock mode runs the whole pipeline with no key. Start the backend. Run inventory to confirm the install. Then run your first experiment with one command. The guide folder in the repository has every command written out.

Slide 13 — Adding your own attack

About 30 seconds

Adding an attack has two paths. Config only takes about two minutes: copy a config, change the target and parameters, then validate, run, evaluate. A brand-new plugin is about fifteen lines: a name, a control point, and one apply function. Register it with two lines, point a YAML at it, and run. I tried this from scratch: validated, run and scored in about ten seconds in mock mode with no key. You never touch the banking agents.

Slide 14 — See it run

About 45 seconds

Now a short live demo of one built-in attack. Step one, open the project. Step two, inventory: it lists the agents, tools and domains MANTIS sees. Step three, run the route-confusion attack against the real system. Step four, evaluate: did it fire, did the outcome change. Step five, open the trace viewer in the browser. Pick the run, load the trace, and you can see the attack event, the scorecard, and which checkpoints fired. The attack fired and the workflow ended completed where manual review was expected. If the model is slow, I have this run recorded and ready.

Slide 15 — Scope, next steps, and questions

About 45 seconds

Scope today: ground truth is a declared baseline plus each plugin's own report, so this is a measurement instrument, not a detector. Results are five trials per configuration on one model, gpt-oss-20b (open-weight), served through UF Navigator, with synthetic data. Next: the main direction is more scenarios, new workflows, attack variants and baselines, added as configs over the same architecture. Then larger campaigns, more models, more institutions, and evaluating other teams' guardrails through the same framework. To sum up: a real system, five control points, plugins in YAML, automatic scoring, and a new person can run an experiment in minutes. Happy to take questions.